Field Hours

# CSCE 716 Foundations and Applications of Blockchains

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

## Three blocks and a broken link

A chain stores, in each block, the hash of the block before it. You can see the mechanism on a toy of 3 blocks. Block 2 stores the hash of block 1. Block 3 stores the hash of block 2. If you change 1 byte in block 1, that hash no longer matches the parent stored in block 2. The chain is no longer intact, even though block 3 still points at the old block 2.

To make every link match after that edit, you must recompute all 3 blocks. Block 2 has to store the new parent hash, which changes block 2's own hash, which forces block 3 to store a new parent too. Copying the old block 3 forward does not hide the edit. The hash is the mechanism, and the number you recompute is the cost. A chain does not make a false block true. It makes a quiet edit expensive.

Links in a three-block toy

Write block 1 -> Store its hash in block 2 -> Store block 2 hash in block 3 -> Recompute all three after an edit

### Change one byte, recompute three

1. You have blocks 1, 2, and 3. Block 2 stores the hash of block 1. Block 3 stores the hash of block 2.

2. You flip one byte in block 1. The parent field in block 2 is now wrong. Block 3 still matches the unchanged block 2.

3. You recompute block 1, then block 2, then block 3. That is 3 blocks. Stopping after block 1 leaves a broken link.

Problem 1. You change a byte in block 1 of a 3-block chain and you do not touch block 2 or block 3. Which link fails?

Answer. The link from block 2 to block 1. Block 2 still stores the old hash. Block 3's link to the unchanged block 2 still matches. The chain as a history of block 1 is not intact.

Problem 2. You want every link to match after replacing block 1. How many blocks must you recompute?

Answer. 3. Block 1 changes, block 2 must store the new hash, and block 3 must store block 2's new hash. Leaving either later block untouched leaves a broken parent.

Lecture 2 · 90 minutes

# A target and an expected number of tries

Proof of work asks you to find a nonce such that the hash of the header falls under a target. In the toy, the first 4 bits of the hash must be 0. If the hash behaves evenly, 1 try in 16 succeeds, so one block expects 16 tries. You do not pick the nonce by taste. The search is the cost that makes a rewrite expensive.

Two independent blocks at that target expect 32 tries, because 16 plus 16 is 32. A header that misses the target is not a finished block. Difficulty is a knob on the target. If you move the toy from 4 leading zero bits to 8, the expected tries for one block jump from 16 to 256. You should compute that change yourself.

Search until the hash fits

1. Set a 4-bit target

2. Hash header plus nonce

3. Stop when 1 of 16 hits

## Expected tries for two blocks

1. The toy target needs 4 leading zero bits. One try succeeds with probability 1 in 16.

2. The expected search for block A is 16 hashes. Block B, independent and at the same target, expects 16 more.

3. Together you budget 32 hashes on average. A header that misses the target is not a finished block.

Problem 1. The target needs 4 leading zero bits, so 1 try in 16 works. What is the expected number of tries for 3 independent blocks?

Answer. 48. Each block expects 16 tries, and 3 times 16 is 48. A header that does not meet the target does not count as one of the successes.

Problem 2. You tighten the toy target from 4 leading zero bits to 8. What happens to the expected tries for one block?

Answer. They rise from 16 to 256. Four zero bits leave 1 success in 16. Eight zero bits leave 1 success in 256. Difficulty is that change in the target.

Lecture 3 · 90 minutes

# A coin you must not spend twice

An unspent-output design treats coins as specific outputs, not as a balance number. A transaction consumes 1 or more outputs and creates new ones. The rule you enforce is simple to state: an output may be consumed once. If two transactions both name output 7, you accept at most 1 of them. The other is a double spend, and every honest node drops it. The chain is the public log of which consumption won.

An account design stores a balance and a sequence number instead. Account 4 with balance 10 can send 3 only if 3 is not greater than 10, and only if the sequence number is the next one you expect. You reject a second transaction that reuses the same sequence number. The two designs differ in what you store, not in the goal.

Either way you need a total order so two conflicting spends cannot both settle.

Two ways to record coins

Unspent outputs

Spend a named output once

Output 7 cannot be reused

Conflicts are explicit

Accounts

Balance plus a sequence

Reject a reused sequence

Overdrafts fail the check

### Output 7 named twice

1. Output 7 is worth 5 units and is still unspent. Transaction A consumes it and creates a new output.

2. Transaction B also names output 7. You already accepted A, so B fails the once-only rule.

3. On an account version of the same story, balance 10 and a repeated sequence number, the second transaction fails the sequence check even if the amount is only 3.

Problem 1. Output 7 is consumed by an accepted transaction. A second transaction names output 7 again. What do you do?

Answer. You reject the second transaction. An output may be consumed once. The chain records which consumption won. You do not merge the two spends.

Problem 2. Account 4 has balance 10 and the next sequence number is 2. A transaction sends 3 with sequence 2, and another sends 3 with sequence 2 again. Which one can settle?

Answer. Only one. The first valid transaction with sequence 2 can move 3, leaving balance 7. The second reuses sequence 2, so you reject it even though 3 is under the old balance.

Lecture 4 · 90 minutes

## Confirmations are a policy, not a proof

A fork is two valid blocks that claim the same parent. Nodes follow the chain with the most accumulated work. A confirmation count is how many blocks sit on top of the one you care about. In a toy where a block takes 10 minutes, 6 confirmations cost 60 minutes of block time. That wait is a policy. It is not a theorem that no competing fork exists.

You do not call a payment final because a wallet printed 6. If one party can produce more work than the honest side, they can grow an alternate chain and overtake those 6 blocks. You state finality as an assumption about hash power plus a confirmation depth you chose. For a 3-block toy with no extra work on top, the tip has 0 confirmations. Depth is a count, and you should be able to point at the blocks you counted.

Depth is blocks built on top

Your block at the start

Later blocks add depth

Six deep is 60 minutes here

Most work picks the fork

### Six blocks at ten minutes

1. Your transaction sits in block 1. Five more blocks are built on it, so you count 5 confirmations so far, not 6.

2. Each block in this toy represents 10 minutes. Five blocks are 50 minutes. You are 10 minutes short of a 6-confirmation policy.

3. A competing fork with less work does not replace your chain. A competing fork with more work does, policy number or not.

Problem 1. In the toy, a block is 10 minutes and your policy is 6 confirmations. How many minutes is that wait?

Answer. 60. Six times 10 is 60. The wait is a policy about depth. It does not prove that a heavier fork cannot appear later.

Problem 2. The tip contains your transaction and nothing is built on it. How many confirmations do you have?

Answer. 0. Confirmations are blocks built on top of the block you care about. The tip itself does not count as one of them in this definition.

Lecture 5 · 90 minutes

## A contract is a state machine

A contract is code plus state that every node runs. You specify the transition before you admire the syntax. For a transfer, you subtract amount M from the sender and add M to the receiver in one step, and you reject the step if the balance would go negative. With balances 10 and 1, a transfer of 4 ends at 6 and 5. A transfer of 11 from the account holding 10 never applies.

You test that spec the way you test any state machine: name the start state, the call, and the end state. Two calls that both spend the same 10 must not both succeed. The chain gives you ordered execution and a public end state. It does not invent a spec you forgot to write. If the code adds M twice and subtracts it once, the chain will faithfully repeat that bug on every node.

One transfer, one transition

1. Read both balances

2. Reject if M does not fit

3. Subtract M and add M

### Balances ten and one

1. Start state: account A holds 10, account B holds 1. The call asks to move 4 from A to B.

2. Four is not greater than 10, so the transition applies. End state: A holds 6, B holds 5.

3. A second call asks to move 11 from A. Eleven is greater than 6, so you reject it and the state stays 6 and 5.

Problem 1. A holds 10 and B holds 1. You apply a valid transfer of 4. What are the two balances afterward?

Answer. A holds 6 and B holds 5. You subtract 4 from 10 and add 4 to 1 in the same transition. The pair sums to 11 before and after.

Problem 2. A holds 6 after that transfer. A call tries to move 11 from A to B. What is the end state?

Answer. Still 6 and 5. The spec rejects a transition that would make the sender negative, so the call does not apply. The chain recording the rejection is not the same as moving the coins.

Lecture 6 · 90 minutes

## What the chain does not know

A chain can order transactions and check the proofs you actually programmed. It does not know the weather, a delivery, or the contents of a box. If your contract pays out when a shipment arrives, some oracle has to write that fact into state. You are trusting that writer for 1 fact the chain cannot see. Naming the oracle is part of the spec. Pretending the chain observed the truck is not.

You also do not get confidentiality from replication. Every node that runs the contract sees the balances in the toy transfer, both the 10 and the 4. Hiding an input takes a different mechanism, and this course's foundation half does not magically add it. If one party can outwork everyone else, your confirmation depth can be replaced. You list these 3 limits next to any claim you make: oracles, visibility, and hash power.

Checks versus outside facts

The chain can check

Order of transactions

Balance rules you coded

Links and work targets

The chain cannot see

A truck or a delivery

Hidden inputs by default

A future heavier fork

### A payout that waits on a truck

1. The contract releases 10 units when a flag becomes 1. Nothing in the block header contains the

truck.

2. An oracle account submits the flag. You name that account in the spec as the trusted writer of 1 fact.

3. Every node sees the flag and the 10. You do not claim the payment was private, and you do not claim a deeper fork is impossible.

Problem 1. Your contract pays 10 units when a shipment arrives, and no oracle is named. What is the hole?

Answer. The chain cannot see the shipment. Some account has to write that fact, and you must name that oracle. Without it, the payout condition is a story, not a transition you can run.

Problem 2. The transfer of 4 from a balance of 10 is replicated to every node. Which of the 3 limits did you fail if you promised the amount stayed hidden?

Answer. Visibility. Replication shows the balances and the amount. Oracles and hash power are separate limits. Hiding inputs is a different mechanism from consensus.

Lecture 7 · 90 minutes

# An escrow with an explicit rule

You can specify a tiny escrow without waving at a platform. The buyer locks 9 units in the contract. Release to the seller requires 2 of 3 signatures: buyer, seller, and arbiter. Refund to the buyer uses the same 2-of-3 rule. A single signature moves 0 units. The state machine is small: locked, released, or refunded, and the terminal states do not accept another spend.

You write the invariant in one line: the contract's balance is 9 until a 2-of-3 release or refund, and then it is 0. You test the boring paths. Buyer plus seller releases. Buyer plus arbiter refunds. Seller alone does nothing. Arbiter alone does nothing. Two releases in a row must fail the second time. If the code pays 9 and leaves 9, the invariant is already false and the chain will keep the bug.

Lock, then a two-of-three spend

Lock nine units -> Collect two signatures -> Pay seller or buyer -> Reject a second spend

## Nine units, two signatures

1. The buyer locks 9. The contract balance is 9. Seller alone submits a release and you move 0.

2. Buyer and seller both sign a release. You pay 9 to the seller and set the contract balance to 0.

3. A second release, even with all 3 signatures, fails. The terminal state does not spend again.

Problem 1. The contract holds 9. Only the seller signs. How many units move, and what is the contract balance?

Answer. 0 units move, and the balance stays 9. Release needs 2 of 3 signatures. One signature is not a transition.

Problem 2. Buyer and arbiter sign a refund, so the buyer receives 9 and the contract holds 0. What

happens if they submit the same refund again?

Answer. The second call fails. The state is already terminal, and the invariant says the balance is 0 after one refund. A second spend would break that invariant.

Lecture 8 · 90 minutes

# What you can now claim a chain does

You can now say what a chain guarantees and stop there. On a toy of 3 blocks, an edit to block 1 forces you to recompute all 3 if every parent hash must match. A 4-bit work target expects 16 tries per block, so the rewrite expects 48 tries. You can reject a second spend of output 7. Six confirmations at 10 minutes is a 60-minute policy, not a proof.

You can also write a spec past a bare transfer. The escrow locks 9 units, moves them only on 2 of 3 signatures, and rejects a second spend. You name any oracle, you admit the state is visible, and you treat hash power as an assumption. If you cannot point at the invariant and at the 3 links you would recompute, you are not ready to claim the application. The chain will run the code you wrote, including the mistake.

Claims you can now support

Parent hashes bind history

Work prices a rewrite

Spend rules are explicit

Oracles stay named

## Rewrite cost on the toy chain

1. Someone wants to replace block 1 and keep a valid 3-block chain. You recompute blocks 1, 2, and 3.

2. Each block expects 16 hashes under the 4-bit target. Three blocks expect 48 hashes.

3. That budget is only the work. It does not make an outside event true, and it does not hide the 9 units in an escrow.

Problem 1. A 3-block toy uses a target of 4 leading zero bits, so each block expects 16 tries. You must recompute every block so the parent links match after an edit to block 1. What is the expected number of tries?

Answer. 48. You recompute all 3 blocks, and 3 times 16 is 48. Fixing only block 1 leaves the parent in block 2 wrong, so the cheaper number 16 is not enough.

Problem 2. Your escrow holds 9 units and your policy waits 6 confirmations at 10 minutes each. Seller alone signs a release. What moves, and how long was the confirmation policy?

Answer. 0 units move, because release needs 2 of 3 signatures. The confirmation policy is 60 minutes, because 6 times 10 is 60. Depth does not relax the signature rule, and one signature does not spend the 9.